Übung 3 Musterlösung

Responsive Web Design

Einführung in Webengineering · WS 2026/27

Diese Musterlösung gehört zur Fassung mit fünf Aufgaben. Aufgabe 4 enthält ein eigenständiges Grid-Beispiel. Andere Lösungen sind möglich, wenn sie die Anforderungen erfüllen.

01 Viewport und Media Queries verstehen

a) Fließende Anpassung und Mobile First

Bei einer fließenden Anpassung verändert sich eine Größe mit dem verfügbaren Platz, etwa eine Breite in `%`. An einem Breakpoint werden zusätzliche Regeln wirksam, beispielsweise für den Wechsel von einer zu zwei Spalten. Beide Ansätze lassen sich kombinieren.

Bei **Mobile First** bilden schmale Ansichten den Ausgangspunkt. Die Basisregeln gelten zunächst für alle Breiten. Eine Media Query mit `min-width` ergänzt oder überschreibt sie ab einer Mindestbreite. Zum Vergleich: Desktop First beginnt mit der breiten Ansicht und verwendet häufig `max-width` für Anpassungen an kleinere Ansichten.

b) Projekt und Viewport

Beide HTML-Dateien liegen direkt in `kapitel03` und binden im `head` dieselbe CSS-Datei ein. Bilder liegen beispielsweise in `kapitel03/images`. Folgende Angaben gehören in beide Dokumente:

```
<link rel="stylesheet" href="css/styles.css">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

`width=device-width` richtet den Layout-Viewport an der Gerätebreite in CSS-Pixeln aus. Mobile Browser sollen keine künstlich breite Desktopansicht verkleinert darstellen. `initial-scale=1.0` legt den anfänglichen Zoomfaktor auf `1` fest; späteres Zoomen bleibt möglich.

c) Ergebnisse der Media Queries

Viewportbreite	Textfarbe	Hintergrundfarbe
375px	gray	#fff4cc
600px	royalblue	#fff4cc
1000px	royalblue	white

`min-width: 600px` bedeutet mindestens `600px`, `max-width: 600px` höchstens `600px`. Beide Grenzen schließen `600px` ein. Dort ändern die Regeln unterschiedliche Eigenschaften und wirken gleichzeitig. Nicht überschriebene Eigenschaften behalten die Werte aus der Grundregel.

02 Flexible Größen und gemeinsame Werte verwenden

a) Bezugsgrößen und berechnete Werte

Bei `width` beziehen sich die `100%` hier auf die Inhaltsbreite des Elterncontainers `body`, die laut Aufgabe der Viewportbreite entspricht. `rem` bezieht sich auf die Schriftgröße des Wurzelelements `html`: `1rem = 16px`. `1vw` entspricht einem Prozent der Viewportbreite.

`--rand: 1rem` ergibt einen Abzug von insgesamt `32px`. `max-width: 60rem` begrenzt die Breite auf `960px`. Die beiden automatischen Außenabstände verteilen den übrigen Platz gleichmäßig.

Viewport	Breite von `.seite`	Abstand je Seite	Schriftgröße `h1`
`375px`	`343px`	`16px`	`23.5px`
`1200px`	`960px`	`120px`	`40px`

Bei `375px`: Breite `375px − 32px = 343px`; Rand `(375px − 343px) / 2 = 16px`; Schrift `16px + 7.5px = 23.5px`.

Bei `1200px`: Die zunächst berechneten `1168px` werden auf `960px` begrenzt. Der Rand beträgt `(1200px − 960px) / 2 = 120px`; die Schrift `16px + 24px = 40px`. Der Wert `--rand` legt damit einen Mindestabstand fest; durch die Maximalbreite kann der tatsächliche Abstand größer werden.

b) Umsetzung und CSS Custom Properties

Der folgende Block bildet die Grundlage der gemeinsamen Datei `css/styles.css`. Die Zahlenprüfung erfolgt unter den Bedingungen aus Teil a).

```
html { font-size: 16px; }
:root { --rand: 1rem; --akzent: #154a6c; }
body { margin: 0; }
.seite {
  width: calc(100% - 2 * var(--rand));
  max-width: 60rem;
  margin: 0 auto;
}
h1 { font-size: calc(1rem + 2vw); color: var(--akzent); }
```

In den Entwicklerwerkzeugen lassen sich die tatsächliche Breite und die Außenabstände im Box-Modell prüfen; die berechnete Schriftgröße steht bei `font-size`. Die Viewportbreite wird in der Geräteansicht auf den gewünschten Wert eingestellt.

`:root` wählt in HTML das Wurzelelement `html` aus. Dort definierte gewöhnliche Custom Properties werden an Nachfahren vererbt. Sie eignen sich damit für gemeinsame Werte. Mit `var(--akzent)` wird der an dieser Stelle geltende Wert von `--akzent` eingesetzt.

Wird in der vorhandenen `:root`-Regel `--akzent: #154a6c` durch `--akzent: #a33b20` ersetzt, wechselt die Überschrift ihre Farbe. Die Regel für `h1` bleibt dabei unverändert.

03 Mit Flexbox ausrichten und Platz verteilen

a) Navigation

Die Navigation enthält eine Liste. Die direkten Listenkinder sind die Flex-Items; die Links befinden sich innerhalb dieser Items.

```
<nav aria-label="Hauptnavigation">
  <ul class="navigation">
    <li><a href="index.html">Start</a></li>
    <li><a href="material.html">Material</a></li>
  </ul>
</nav>
```

```
.navigation {
  display: flex; flex-direction: column; gap: 0.5rem;
  list-style: none; margin: 0; padding: 0;
}
.navigation a { display: block; padding: 0.5rem; }
@media (min-width: 600px) {
  .navigation {
    flex-direction: row; flex-wrap: wrap;
    justify-content: flex-end;
  }
}
```

Die Liste `.navigation` ist der **Flex-Container**, ihre `li`-Elemente sind die **Flex-Items**. In der üblichen horizontalen Schreibrichtung verläuft die Hauptachse bei `column` von oben nach unten und bei `row` von links nach rechts. `justify-content: flex-end` richtet die Items am Ende dieser Achse aus. `flex-wrap: wrap` erlaubt ab `600px` zusätzliche Zeilen.

b) Basisgröße sowie Wachsen und Schrumpfen

`flex: 1 1 150px` steht für `flex-grow: 1`, `flex-shrink: 1` und `flex-basis: 150px`. Die Basis ist die Ausgangsgröße vor der Verteilung von freiem oder fehlendem Platz. Der Grow-Faktor verteilt zusätzlichen Platz; der Shrink-Faktor steuert das Schrumpfen bei Platzmangel.

Container	Rechnung	Breite je Karte
`750px`	`750 − 3 × 150 = 300` frei `150 + 300 / 3`	`250px`
`360px`	`3 × 150 − 360 = 90` fehlen `150 − 90 / 3`	`120px`
`360px`, kein Schrumpfen	`3 × 150 = 450` benötigt	`150px`

Die gleiche Verteilung beim Schrumpfen ergibt sich hier aus gleichen Basisgrößen und gleichen Faktoren. Allgemein zählt das Produkt aus `flex-shrink` und Basisgröße. Mit `flex-shrink: 0` bleiben die Karten zusammen `450px` breit und ragen `90px` über den `360px` breiten Container hinaus.

Zum Prüfen werden die Regeln der Aufgabe auf einen Container mit drei direkten `div`-Kindern angewendet. In den Entwicklerwerkzeugen ändern Sie die Containerbreite und anschließend `flex-shrink`. Abstände oder Rahmen würden die vereinfachte Rechnung verändern.

04 Mit Grid ein responsives Seitenraster erstellen

a) Eigenständige Beispielseite

Im Ordner `aufgabe04` liegen `index.html` und `styles.css`. Die HTML-Datei erhält ein Grundgerüst mit `<!doctype html>`, `<html lang="de">`, `head` und `body`. Im `head` stehen:

```
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Responsives Seitenraster</title>
<link rel="stylesheet" href="styles.css">
```

Im `body` steht die vorgegebene Struktur mit den fünf direkten Kindern von `.layout`. Der Link verweist auf `main` mit der ID `inhalt`; weiterer HTML-Code ist nicht erforderlich.

b) Einspaltiges Raster und Wechsel bei 900px

Die vollständigen Layoutregeln in `styles.css` lauten:

```
body { margin: 0; }
.layout {
  display: grid; margin: 16px; gap: 16px;
  grid-template-columns: 1fr;
  grid-template-areas: "kopf" "menue" "inhalt" "info" "fuss";
}
.layout > header { grid-area: kopf; }
.layout > nav    { grid-area: menue; }
.layout > main   { grid-area: inhalt; }
.layout > aside  { grid-area: info; }
.layout > footer { grid-area: fuss; }
.layout > * { min-width: 0; overflow-wrap: anywhere; }
@media (min-width: 900px) {
  .layout {
    grid-template-columns: 2fr 1fr;
    grid-template-areas: "kopf kopf" "menue menue"
                         "inhalt info" "fuss fuss";
  }
}
```

Unter `900px` entsteht eine Spalte mit fünf Zeilen. Ab `900px` stehen `main` und `aside` nebeneinander; die übrigen Bereiche reichen über beide Spalten. `min-width: 0` und `overflow-wrap: anywhere` verhindern, dass lange Inhalte die Spalten verbreitern. Die `16px` Außenabstand zum Browserrand sind frei gewählt.

c) Spaltenbreiten und Bedeutung von fr

Die `916px` sind die Inhaltsbreite des Grid-Containers, nicht die Viewportbreite. Nach Abzug des Spaltenabstands bleiben `916px − 16px = 900px`. Die drei Anteile ergeben `1fr = 300px`. Die Spalten sind daher `600px` und `300px` breit.

`fr` bezeichnet einen Anteil am für flexible Rasterspalten verfügbaren Platz. Grid eignet sich hier, weil Zeilen und Spalten gemeinsam festgelegt werden und einzelne Bereiche mehrere Spalten überspannen können.

05 Bilder anpassen und das Ergebnis testen

a) Bildauswahl und flexible Darstellung

Die folgenden Dateinamen sind Beispiele für zwei vorhandene Varianten desselben Motivs. Sie werden durch die eigenen Bilddateien ersetzt. Der Alternativtext beschreibt deren Inhalt.

```
<picture>
  <source media="(max-width: 600px)" srcset="images/schmal.jpg">
  <img src="images/breit.jpg" alt="Treffen der Lerngruppe">
</picture>
```

```
img { display: block; max-width: 100%; height: auto; }
```

Bis einschließlich `600px` passt die Bedingung des `source`-Elements: Der Browser verwendet `images/schmal.jpg`. Oberhalb dieser Breite verwendet er `images/breit.jpg` aus dem `img`. Dieses Element stellt auch den Alternativtext und die Rückfalllösung bereit.

`max-width: 100%` begrenzt das Bild auf die Inhaltsbreite seines Containers. Ohne feste Breite oder `width: 100%` wächst es nicht über seine natürliche Breite hinaus. `height: auto` erhält das Seitenverhältnis. Die Bildauswahl ist unabhängig von dieser Skalierung.

b) Erwartetes Verhalten und Beispielbeobachtungen

Die folgenden Beobachtungen beziehen sich auf die eigene Webseite in `kapitel03` und auf das separate Grid-Beispiel in `aufgabe04`. Beide Seiten der Webseite und das Grid-Beispiel sind im Browser zu prüfen.

Viewport	Erwartete Beobachtung	Verantwortliche Regel
`375px`	Im Grid-Beispiel stehen alle fünf Bereiche untereinander.	Basisraster mit `1fr` und fünf Zeilen in `grid-template-areas`.
`768px`	Die Navigation liegt rechts in einer Zeile, sofern die Links hineinpassen.	`min-width: 600px`, `flex-direction: row` und `justify-content: flex-end`.
`1200px`	Im Grid-Beispiel stehen Hauptinhalt und Zusatzinformationen nebeneinander.	`min-width: 900px` und `grid-template-columns: 2fr 1fr`.

Grenzwerte: Auf der Webseite steht die Navigation bei `599px` untereinander, bei `600px` und `601px` nebeneinander. Das schmale Bild gilt noch bei `600px`, das breite erst darüber. Das separate Grid-Beispiel bleibt bei `899px` einspaltig und wird bei `900px` und `901px` zweispaltig.

Zoom: Bei `200%` Browserzoom müssen Texte und Bedienelemente lesbar und erreichbar bleiben. Zoom kann die in CSS-Pixeln verfügbare Viewportbreite verkleinern und dadurch ein schmaleres Layout aktivieren. Entscheidend ist, dass Inhalte nicht überlappen oder abgeschnitten werden.

Prüfung: Kontrollieren Sie auch die Bildauswahl, das Seitenverhältnis und die Seitenbreite. Im Elements-Bereich kann am ausgewählten `img` die aktuell geladene Ressource geprüft werden. Falls horizontales Scrollen auftritt, suchen Sie zuerst nach festen Breiten, übergroßen Medien oder langen untrennbaren Inhalten. Ein bloßes Abschneiden des Überlaufs behebt die Ursache nicht.

Beide Seiten in `kapitel03` verwenden dieselbe Datei `css/styles.css` mit den Regeln aus Aufgaben 2, 3 und 5. Aufgabe 4 hat eine eigene HTML- und CSS-Datei. Die Beispielregel `.hinweis` und der Rechenversuch mit `.karten` werden für das Seitenlayout nicht benötigt.